Designing a Model for improving CPU Scheduling by using Machine Learning

Naila Aslam[1], Nadeem Sarwar[2], Amna Batool[3]
Department of Computer Sciences & IT
KFUEIT, Rahim Yar Khan, Pakistan[1], University of Gujrat Sialkot Campus, Sialkot, Pakistan[2]
Nailaaslam163@gmail.com[1], Nadeem_srwr@yahoo.com[2], Amnarustam24@gmail.com[3]

Abstract—**In this paper, we have proposed a model that will help in improving the CPU scheduling of a uni-processor system. The model will use Bayesian Decision Theory as classifier tool that will select an appropriate process from a ready queue. This selection process consists of a two phases; in the first phase its compares the static and dynamic properties of the new processes in the queue with the properties of dataset of the previously executed processes. The dataset is divided into two categories of processes; useful processes and not-useful processes. The new process will be categorized as either useful or not-useful depending on the results of comparison of properties. Our proposed model is applicable in the low-level language such as Assembly at kernel level. Unix/Linux are the better platform for the implementation of our model because Linux/Unix environments are open source and their kernels are editable.**

Keywords. CPU, scheduling, Machine learning, Model, Processes, OS.

I. INTRODUCTION

Only one process will run at a time on a uniprocessor operating system. Other processes must hold up in series, until the CPU is free for the awaiting processes which can be rescheduled. The objective of multiprogramming is to have a numerous processes running at all times, to increase the CPU usage. A process is executed until it must hold up, typically for the completing of some I/O request. In a simple machine process, the CPU then just sits unmoving. All the waiting time is wasted; no useful work is done. With multiprogramming, we will try to use this time beneficially. A couple of processes are kept in memory at one time.

CPU Scheduler

Whenever the CPU does nothing, the operating system must select one of the processes in the ready queue to run. The selection process is maintained by the short-term scheduler, or CPU scheduler.

Scheduling Criteria

Different CPU-scheduling algorithms have dissimilar properties, and the choice of a particular algorithm may favor one class of processes over alternative. To choose which algorithm to use in a particular condition, we must consider the properties of the several algorithms. Many criteria have been recommended for comparing CPU-scheduling algorithms. Which characteristics are used for comparison can make a significant difference in which algorithm is judged to be best. The criteria include the following:

CPU Utilization, Throughput, Response Time, Turnaround, Time, Waiting Time

So, a good scheduling algorithm for real time and time sharing system are concluded that must possess following characteristics: Minimum context switches, Maximum CPU utilization, Maximum throughput, Minimum turnaround time, Minimum waiting time.

There are many different CPU-scheduling algorithms.

- First-Come, First-Served Scheduling (FCFS)
- Priority Scheduling
- Round-Robin Scheduling
- Multilevel Queue Scheduling

II. RELATED WORK

Most datacenters, mists and networks comprise of various eras of processing frameworks, each with distinctive execution profiles, representing a test to occupation schedulers in accomplishing the best utilization of the foundation. A helpful bit of data for scheduling employments, normally not accessible, is the degree to which applications will utilize accessible assets once they are executed. [1]

A Linux system, Machine Learning (ML) methods are used to study the behavior of programs and CPU time slice utilization. Learning is carried out by an analysis of certain dynamic and static attributes of the processes while they are being run. Their goal was to determine the most important dynamic and static attributes of the processes that can help in calculation of CPU burst times which minimize the process TaT (Turn-around-Time). In their research they modify the Linux Kernel scheduler (version 2.4.20-8) to allow scheduling with customized time slices. [2]

III. MATERIAL AND METHODS

A. Bayesian Decision Theory

Programming computers to make inference from data is a cross between statistics and computer science, where statisticians provide the mathematical framework of making inference from data and computer scientists work on the efficient implementation of the inference methods. Data comes from a process that is not completely known.

https://sites.google.com/site/ijcsis/
ISSN 1947-5500

The extra pieces of knowledge that we do not have access to are named the unobservable variables. In the coin tossing example, the unobservable variables are its initial position, the force and its direction that is applied to the coin when tossing it, where and how it is caught, and so forth and the observable event the outcome of the toss. Denoting the un-observables by z and the observable as x, in reality we have

$$x=f(z) \quad (1)$$

Where f(·) is the deterministic function that defines the outcome from the unobservable pieces of knowledge. Because we cannot model the process this way, we define the outcome X as a random variable drawn from a probability distribution P (X = x) that specifies the process.

The outcome of tossing a coin is heads or tails, and we define a random variable that takes one of two values. Let us say X=1 denotes that the outcome of a toss is heads and X=0 denotes tails. Such X are Bernoulli-distributed where the parameter of the distribution po is the probability that the outcome is heads:

$$P(X = 1) = po \text{ and } P(X = 0) = 1 – P (X = 1) = 1 – po \quad (2)$$

Assume that we are asked to predict the outcome of the next toss. If we know po, our prediction will be heads if po > 0.5 and tails otherwise.

This is because if we choose the more probable case, the probability of error, which is 1 minus the probability of our choice, will be minimum.

If this is a fair coin with po = 0.5, we have no better means of prediction than choosing heads all the time or tossing a fair coin ourselves. If we do not know P(X) and want to estimate this from a given sample, then we are in the realm of statistics. We have a sample, X, containing sample examples drawn from the probability distribution of the observables x^t, denoted as p(x). The aim is to build an approximator to it, ˆ p(x), using the sample X.

In the coin tossing example, the sample contains the outcomes of the past N tosses. Then using X, we can estimate po, which is the parameter that uniquely specifies the distribution. Our estimate of po is

$$po = \#\{\text{tosses with outcome heads}\} / \#\{\text{tosses}\} \quad (3)$$

Numerically using the random variables, x^t is 1 if the outcome of toss t is heads and 0 otherwise. Given the sample {heads, heads, heads, tails, heads, tails, tails, heads, heads}, we have X = {1, 1, 1, 0, 1, 0, 0, 1, 1} and the estimate is

$$po = \frac{\sum_{t=1}^{N} x^t}{N} \quad (4)$$

B. Proposed Approach

Our purpose in the proposed model is to reduce the possibility of selecting an inappropriate process that may increase the waiting time of all other processes waiting for CPU. Furthermore, (throughput) will be decreased on selecting the process which will take maximum time of CPU. The selection of such a process should be carefully performed so that we could attain almost all the criteria of CPU scheduling. One of the frequently used technique is Bayesian Decision Theory (BDT), which works on previous knowledge and distribution of the data from which we have to select the appropriate data item expecting to achieve the target.

Our model proposed the data set of 100 execution instances of five programs: (1) matrix multiplication, (2) quick sort, (3) merges sort, (4) heap sort and (5) a recursive Fibonacci number generator. The data collection may be performed by saving the process control blocks of the executed processes. The collected data would be of 5 programs with different input sizes and different best TaT. Data of about 100 instances of the five programs is enough and made into 02 categories; useful and not-useful processes; based on the attribute TaT class with each class having an interval of 50 ticks.

Training and Testing methodology: We proposed two types of tests on the training examples with all the learners described in the section, BDT will be applied as classifier, on the data sets collected in the first phase. The tests are:

Use Training Set: The classifier is evaluated on how well it predicts the class of the instance it was trained on.

Cross-Validation: The classifier can be evaluated by cross-validation, using the number of processes that are entered in the system. Recognition accuracy can be tested via cross-validation.

C. The Design of Modified Scheduling Process

Following is the detail of the design of modified scheduling process and the steps to minimize TaT of a program. The steps to minimize TaT of a program are as shown below in numbers from 1 to 5.

1) The program 'X' is given to BDT as an input.

Extracting the best attributes is nothing but feature selection

2) The BDT will classify 'X' and output its expected class.

3) We send this information to modified scheduler through a system call.

4) The scheduler instructs the CPU such that CPU allocates clock ticks to 'X'.

5) After allocating ticks to 'X' it will run with minimum TaT.

Overall scheme of our methodology is explained as follows:

1) Run the programs with different time slices with modified O(1) scheduler and find STS (best special time slice) which gives minimum turn-around-time (TaT).

2) Build the knowledge base of static and dynamic characteristics of the programs from the run traces obtained in step 1 and train them with the BDT algorithm.

3) If a new program comes, classify it and run the program with this predicted STS.

4) If the new program instance is not in the knowledge-base, go to step 1.

https://sites.google.com/site/ijcsis/
ISSN 1947-5500

Mathematically Gaussian is described as:

$$f(x) = \frac{1}{\sqrt{2\pi}} e^{-\frac{(\mu - x)^2}{2\sigma^2}} \quad (5)$$

where μ is mean and σ is variance.

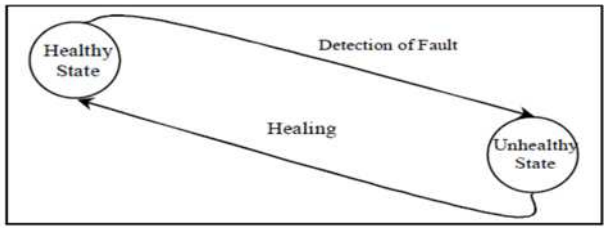

Fig 1: *State chart of self-selecting system detecting*

By considering the static and dynamic characteristics of a program, we can schedule it using modified scheduler such that the turn-around-time of it is minimized. We show that machine learning algorithms are very efficient in the process characterization process. The BDT algorithm will achieved good prediction (91% -- 94%), which indicates that when suitable attributes are used, a certain amount of predictability does exist for known programs. Our proposed approach will expect that 4% to 8% reduction in turn-around-time is possible and this reduction rate slowly increases with the input size of the program. We find the best features: input size, program size, global data container, local text, read only segment of the process and input type of a program that can characterize its execution behavior. We conclude that our technique can improve the scheduling performance in a single system.

We have designed only a prototype of self-selecting operating system based on the proposed architecture keeping in mind the Linux kernel as our kernel –A hypervisor virtual machine– on Linux as the hypervisor layer's micro-kernel. We moved up scheduler drivers as daemons and placed the driver polling middleware to handle devices and communicate to the kernel. We added all operating system resource control mechanisms, like deadlock manager, paging manager, and lock manager, to the super-selecting unit.

The architecture and model of our proposed approach is given below.

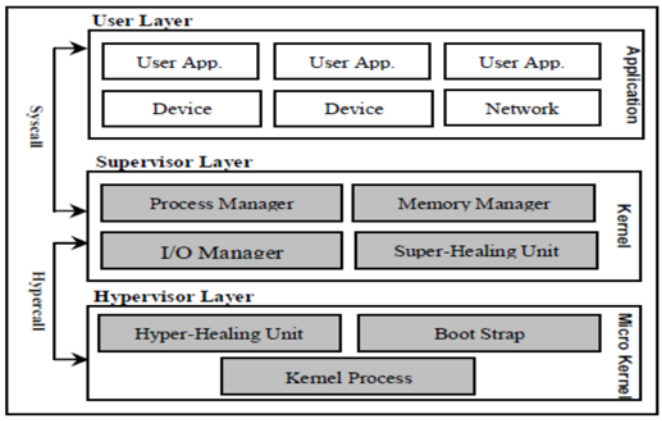

Fig 2: *Framework of the proposed model*

The internal structure of the modification of our proposed system is given below.

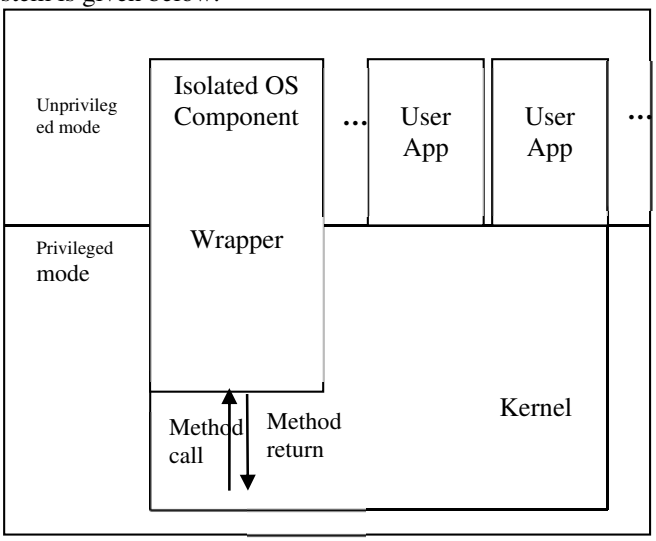

Fig 3: *Wrapper module isolating the kernel modules from our proposed module.*

The above figure adds a component wrapper that will allow our application (at user level) to interact the scheduling procedure dispatcher. The component will be invoked by system call at user level and allows the user to select the appropriate process by involving kernel level modules. The logic and methodology of our proposed module is implemented at supervisor layer and transparent from user. The beauty behind the logic we proposed is that the selection of the process is solely on the kernel shoulders. If the selected process is classified as incorrect, the previous history is updated by adding the incorrect process. The proposed scheduler can make the decisions up to 96% accurate.
The decision is made by comparing following dynamic and static properties of a process.

Attribute	Definition
Has	This is symbol hash size in bytes.
Dynamic	This dynamic linking information size in bytes.
dynsir	This is size (bytes) of strings needed for dynamic linking.
Msh	This is symbol has table size bytes.
Dynamic	This dynamic linking information size in bytes.
Dynsir	This is size (bytes) of strings needed for dynamic linking, most commonly the strings that represent the names associated with symbol table entries.
Dynsym	This is size (bytes) of the dynamic linking symbol table.
Got	This is the global offset table size in bytes.
Pit	This is the procedure linkage table size in bytes.
RoData	This is read-only data size (bytes) that typically contribute to a non-writable segment in the process image.
Ryl.Dyn	This is the size (bytes) of the relocation information size.
Text	This is the "text" or "executable instructions", size (bytes) of a program.
Data	This is the size (bytes) of the initialized data that contribute to the size of the program's memory image size.
Bss	This section holds uninitialized data size.
Total-size	This is total size (bytes) of the program.
Program_Name	This is name of the program and a nominal attribute.

Fig 3: *Dynamic and static properties of a process with their definitions*

https://sites.google.com/site/ijcsis/
ISSN 1947-5500

D. Limitations of our Method

The characteristics of the programs and the gain in the turnaround time depends on the architecture and operating system. Our modified scheduler is not designed with any security features that would prevent a user from writing their own process and requesting INT MAX processor time via system call. Setting p > special time slice to INT MAX could significantly slow down a system.

IV. Results and Discussion

Scheduling policy, such as run-time priority-based preemption or a single memory allocation policy recognize that differing application requirements may best be solved with differing task scheduling or memory allocation policies. Future real-time kernels will have modular, replaceable memory policy sub-components and task scheduling policy sub-components. In this way, operating system policies can be changed during software design or as application complexity grows, without having to discard and replace the entire kernel and application ties to it which have already been enveloped. Figure 1 illustrates the Modularity and Interchangeability of O/S Policies. For example, early in a software design variable block sized memory allocation may be used. But later, the memory fragmentation implicit in this approach and its associated lack of determinism, may become critical to the software architect. It may then want to change to another memory allocation approach, based on multiple partitions of fixed-size blocks. It will be able to do this by simply exchanging the memory policy sub-component, without replacing his kernel and endangering his investment in existing application software. Similarly, a designer will be able to choose from a broadly-based library of task scheduling policy sub-components, supporting paradigms such as round-robin (sequential), priority-based preemption, rate monotonic scheduling, deadline scheduling, off-line pre-scheduling, and others. Given below are the future targets that can be achieved by modifying the kernel routines and ML techniques in both standalone and parallel systems.

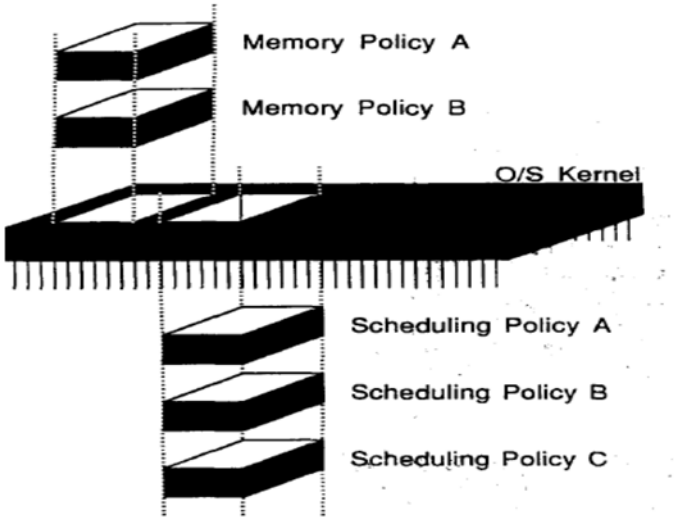

Fig 5: Modularity and Interchangeability of SO Policies

V. Conclusion

A variety of criteria are used in designing the real-time scheduler. Some of these criteria relate to the behavior of the system as perceived by the individual user (user oriented), while others view the total effectiveness of the system in meeting the needs of all users (system oriented). Some of the criteria relate specifically to quantitative measures of performance, while others are more qualitative in nature. From a user's point of view, response time is generally the most important characteristic of a system, while from a system point of view, throughput or processor utilization is important.

In this work, BDT works as effective classifier to classify the process which may or may not be useful process for the system from both user and system point of view. The BDT is solely based on probabilistic and statistical data so as a result the ratio of accuracy of the selecting the appropriate process may vary from time to time.

References

[1]. Matsunaga, Fortes.; On the use of machine learning to predict the time and resources consumed by applications.10th IEEE/ACM International Conference on Cluster, Cloud and Grid Computing, 2010.

[2]. Muhsen, Babiceanu.; Systems Engineering Approach to CPU Scheduling for Mobile Multimedia Systems - 2011 IEEE International Systems Conference, (SysCon), 2011.

[3]. Alam et al.; Finding Time Quantum of Round Robin CPU Scheduling Algorithm Using Fuzzy Logic. International Conference on Computer and Electrical Engineering, 2008.

[4]. Li, Fan.; Live Migration of Virtual Machine Based on Recovering System and CPU Scheduling. Information Technology and Artificial Intelligence Conference (ITAIC), 2011 6th IEEE Joint International, 2011.

[5]. Choi, Yun.; Context Switching and IPC Performance Comparison Between uClinux and Linux on the ARM9 based Processor. SAMSUNG Tech. Conference 2012.

[6]. Pham et al.; A Simulation Framework to Evaluate Virtual CPU Scheduling Algorithms. IEEE 33rd International Conference on Distributed Computing Systems Workshops, 2013

[7]. Jawad.; Design and Evaluation of a Neuro-fuzzy CPU Scheduling Algorithm. 2014 IEEE 11th International Conference on Networking, Sensing and Control (ICNSC), 2014.

[8]. Berral et al.; Adaptive Scheduling on Power-Aware Managed Data-Centers using Machine Learning. 12th IEEE/ACM International Conference on Grid Computing, 2011.

[9]. Punhani et al.; A Cpu scheduling based on multi criteria with the help of Evolutionary Algorithm. 2nd IEEE International Conference on Parallel, Distributed and Grid Computing 2012.

[10]. BAJWA, I., & SARWAR, N. (2016). AUTOMATED GENERATION OF EXPRESS-G MODELS USING NLP. Sindh University Research Journal-SURJ (Science Series), 48(1).(pp. 05-12)

https://sites.google.com/site/ijcsis/
ISSN 1947-5500